10-10-2026

INSTITUTO TECNOLÓGICO DE LAS AMÉRICAS (ITLA)

Asignatura: Introducción a la Programación

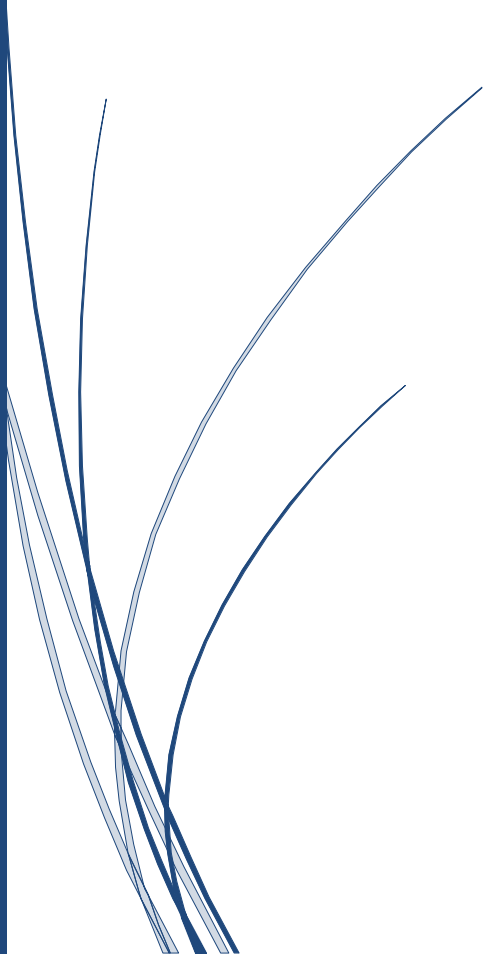
TAREA CAPÍTULO IX: ARREGLOS Y VECTORES

Nombre: Amauri Feliz Valenzuela

Matrícula: 201631629

Grupo o sección: PRG-101-C-2026-03-JG

Profesor: Jimmy Gómez



Índice

Introducción 3

Objetivos 3

9.1 ¿Qué es un arreglo? 3

9.2 Índices y subíndices 4

9.3 Vectores y características 4

Desarrollo del proyecto 5

Explicación del código 6

REPOSITORIO DEL PROYECTO 7

Evidencias de ejecución 8

Casos de prueba 16

Conclusión 17

Fuentes consultadas 18

Introducción

Organizar varias calificaciones permite consultar resultados sin mantener una variable diferente para cada estudiante. Este trabajo estudia los arreglos, los índices y los vectores mediante un sistema de consola en Python. El ejemplo reúne cinco nombres y cinco notas, ofrece un menú y comprueba que las entradas se encuentren dentro de los límites establecidos.

Las explicaciones corresponden al código entregado y las pruebas se apoyan en transcripciones reales. La ejecución se realizó con Python 3.12.14 desde una consola automatizada en Windows. Las siete capturas aportadas por el estudiante muestran además el código y la ejecución manual en Visual Studio Code. Las pruebas automatizadas y las capturas constituyen evidencias distintas y sus resultados coinciden.

Objetivos

El objetivo general es demostrar cómo almacenar y procesar información con listas que representan vectores unidimensionales.

Los objetivos específicos son relacionar nombres y notas por sus índices; consultar y modificar posiciones; recorrer listas con ciclos; calcular resultados; aplicar condiciones para clasificar estudiantes; validar entradas y comprobar el programa con casos reproducibles.

9.1 ¿Qué es un arreglo?

Un arreglo reúne varios elementos bajo un mismo nombre y permite consultar sus posiciones. Se utiliza para manejar conjuntos como nombres, temperaturas o calificaciones. En lugar de repetir una operación para muchas variables separadas, se puede recorrer el conjunto. Esto facilita organizar los datos y evita escribir instrucciones similares una y otra vez.

Una variable individual como nota = 85 guarda una sola calificación. Una lista como la siguiente reúne cinco:

```
nota = 85
calificaciones = [85, 90, 78, 95, 88]
print(nota)
print(calificaciones[0])
```

Este ejemplo educativo representa las calificaciones de cinco estudiantes. Contiene cinco elementos y almacena números enteros. Ambas instrucciones de impresión muestran 85, pero la segunda consulta una posición dentro de una colección. Puedo consultar otra posición o recorrer las cinco notas para procesarlas. Estos valores son un ejemplo conceptual; el programa práctico utiliza las notas 85, 67, 92, 74 y 58. Una lista también puede almacenar texto o valores lógicos. Para este ejercicio se separan los nombres, que son cadenas, y las notas, que son enteros.

Conviene distinguir términos: muchos lenguajes tienen arreglos de tamaño fijo y elementos del mismo tipo. Las listas de Python pueden cambiar de tamaño y admitir tipos diferentes. Aquí se utilizan como una representación práctica de arreglos, sin afirmar que todas sus propiedades sean iguales. Las ventajas se observan al recorrer las cinco notas con una sola secuencia de instrucciones.

9.2 Índices y subíndices

El índice identifica una posición. En la expresión nombres[2], el número dentro de los corchetes actúa como subíndice y permite acceder al tercer elemento. En este contexto se usan ambos términos para hablar del acceso a una posición.

Python numera las posiciones desde cero por convención. Una forma de comprenderlo es contar cuántas posiciones se avanza desde el comienzo: para el primer elemento no se avanza ninguna. No todos los lenguajes empiezan en cero. Una lista de cuatro elementos tiene longitud 4 y último índice no negativo 3; en general, ese último índice es la longitud menos uno cuando la lista no está vacía.

```
nombres = ["Ana", "Luis", "Carlos", "María"]
print(nombres[0])
print(nombres[2])
nombres[1] = "Pedro"
```

La primera impresión muestra Ana y la segunda Carlos. La última instrucción cambia Luis por Pedro y no imprime nada por sí misma. Después, la lista queda como ["Ana", "Pedro", "Carlos", "María"]. Consultar nombres[4] produce IndexError, porque esa posición no existe. El programa valida el índice antes de acceder a las listas para evitar ese error.

Python también admite índices negativos; por ejemplo, -1 identifica el último elemento. El menú de este proyecto acepta únicamente índices de 0 a 4 para mantener una numeración clara y evitar confundir una entrada negativa con una consulta válida.

9.3 Vectores y características

En programación introductoria, un vector es un arreglo unidimensional. Unidimensional significa que se necesita un solo índice para ubicar un elemento. No es una tabla de filas y columnas. En este trabajo cada lista es una secuencia ordenada, accesible por posición, recorrible y modificable.

Los elementos conservan un orden y cada posición tiene un índice. Un vector permite almacenar varios valores, consultarlos, modificarlos y recorrerlos en secuencia. Para este ejercicio introductorio, las listas de Python representan vectores unidimensionales; no son arreglos tipados de otras bibliotecas.

El ciclo for visita un elemento en cada iteración:

```
edades = [18, 21, 19, 25, 20]
for edad in edades:
    print(edad)
```

En la primera iteración, edad toma el valor 18 y lo imprime. En la segunda toma 21; en la tercera, 19; en la cuarta, 25; y en la quinta, 20. La salida son esos cinco números, uno por línea. Al terminar los elementos, el ciclo finaliza.

También puedo recorrer el mismo vector con while:

```
edades = [18, 21, 19, 25, 20]
indice = 0
while indice < len(edades):
    print(edades[indice])
    indice += 1
```

Primero inicializo indice en 0. Antes de cada repetición evalúo si es menor que len(edades), que vale 5. Si la condición es verdadera, consulto edades[indice] y lo imprimo. Después aumento el índice en uno. Se visitan las posiciones 0, 1, 2, 3 y 4, y se muestran 18, 21, 19, 25 y 20. Cuando el índice llega a 5, la condición es falsa y el ciclo termina. El incremento evita que se repita indefinidamente la misma posición.

Estos dos ejemplos pertenecen a la investigación. En main.py, for recorre las listas y while mantiene activo el menú y repite la lectura de entradas incorrectas.

Desarrollo del proyecto

El problema consiste en consultar y actualizar las calificaciones de cinco estudiantes sin perder la relación entre una persona y su nota. Un sistema pequeño puede ayudar a organizar un ejercicio académico, calcular resultados rápidamente y reducir errores al introducir datos.

La solución propuesta utiliza dos listas del mismo tamaño y orden. El elemento 0 de estudiantes es Ana y el elemento 0 de calificaciones es 85. Si se cambiara el orden de una sola lista, se podrían asignar notas a personas equivocadas. Por eso el programa modifica únicamente la nota en la posición seleccionada y no ordena las listas por separado.

Los nombres Ana, Luis, Carlos, Maria y Pedro y las notas 85, 67, 92, 74 y 58 son datos de ejemplo indicados en el enunciado. Las pruebas ejecutan ese conjunto y modificaciones de demostración. No se consultaron expedientes académicos ni bases de datos de estudiantes para obtenerlos.

La escala permitida va de 0 a 100 y admite enteros. El mínimo de aprobación es 70, una regla definida exclusivamente para este proyecto. Los datos permanecen en memoria: al cerrar se pierden las modificaciones y al iniciar otra sesión vuelven los valores originales. No se utilizan archivos de almacenamiento, bases de datos ni interfaces gráficas en la aplicación.

Herramientas y funcionamiento

El archivo main.py contiene la aplicación y utiliza funciones sencillas. Primero se prepararon las listas y el menú; luego se ejecutaron sesiones con entradas controladas y se comprobaron las salidas. Los registros conservan tanto las entradas enviadas como el texto producido y el código de salida del proceso.

Python ejecuta el código. Visual Studio Code es el editor utilizado para las capturas del código y la terminal integrada. La terminal de Windows permite iniciar el programa. Git sirve para registrar versiones y GitHub para alojar y compartir el repositorio. El repositorio público de GitHub se publicó y su acceso se verificó sin credenciales. Las capturas documentan la ejecución realizada por el estudiante.

La opción 1 muestra nombres, notas e índices. La 2 consulta una posición. La 3 calcula el promedio; 4 y 5 muestran máximo y mínimo. La 6 muestra y cuenta aprobados y la 7 hace lo mismo con reprobados. La 8 cambia una nota y muestra el valor anterior y el nuevo. La 9 busca un nombre sin distinguir mayúsculas y quitando espacios externos. La 10 muestra el resumen actualizado. La opción 0 termina el programa.

leer_entero convierte la entrada mediante int. Si hay letras o un decimal, captura ValueError y solicita otro entero. Si el valor queda fuera del rango, muestra los límites y repite la pregunta. Un índice incorrecto no llega a utilizarse para acceder a la lista. Las interrupciones de entrada y de teclado también reciben un mensaje de cierre.

Explicación del código

1. Creación de los vectores

```
estudiantes = ["Ana", "Luis", "Carlos", "Maria", "Pedro"]
calificaciones = [85, 67, 92, 74, 58]
```

Cada lista almacena cinco elementos. El resultado esperado al mostrar ambas listas es Ana 85, Luis 67, Carlos 92, Maria 74 y Pedro 58. Esto aplica el almacenamiento de varios datos y la correspondencia por posición estudiada en el capítulo.

2. Acceso mediante índices

```
indice = leer_entero("Índice del estudiante (0 a 4): ", 0, 4)
print(f"{estudiantes[indice]}: {calificaciones[indice]} puntos")
```

Después de validar la posición se consulta el mismo índice en las dos listas. Al introducir 2 se muestra Carlos: 92 puntos. El índice permite recuperar una pareja de datos sin recorrer manualmente las posiciones anteriores.

3. Recorrido mediante ciclos

```
for indice in range(len(estudiantes)):
    print(f"{indice}. {estudiantes[indice]}: {calificaciones[indice]} puntos")
```

len obtiene cinco y range(5) genera los índices 0, 1, 2, 3 y 4. El ciclo repite la impresión para cada uno. El resultado son las cinco líneas del listado inicial. Se relacionan aquí el recorrido del vector y el acceso mediante subíndices.

4. Condiciones y conteo

```
aprobo = calificaciones[indice] >= MINIMO_APROBACION
if aprobo == aprobados:
    print(f"{estudiantes[indice]}: {calificaciones[indice]} puntos")
    cantidad = cantidad + 1
```

La comparación determina si una nota llega a 70. El parámetro aprobados indica qué grupo se quiere mostrar: True para aprobados y False para reprobados. Cada coincidencia aumenta el contador. Inicialmente hay tres aprobados y dos reprobados. Las condiciones permiten decidir qué elementos cumplen una regla; if, elif y else también seleccionan las operaciones del menú.

5. Cálculo del promedio

```
total = 0
for nota in calificaciones:
    total = total + nota
return total / len(calificaciones)
```

El acumulador empieza en cero y suma cada calificación. La suma inicial es 85 + 67 + 92 + 74 + 58 = 376; 376 / 5 = 75.2. El formato de impresión muestra 75.20. Se combinan recorrido y operaciones matemáticas. La función supone una lista no vacía; en esta aplicación siempre existen cinco notas.

6. Modificación

```
anterior = calificaciones[indice]
calificaciones[indice] = nueva
print(f"{estudiantes[indice]}: antes {anterior}, ahora {nueva} puntos")
```

Antes de este fragmento se validan el índice y la nueva nota. Para el índice 1 y el valor 80, la salida indica Luis: antes 67, ahora 80 puntos. La lista queda [85, 80, 92, 74, 58] y el promedio pasa a 77.80. La asignación sustituye un elemento y conserva su correspondencia con el nombre.

7. Búsqueda

```
for indice in range(len(estudiantes)):
    if estudiantes[indice].lower() == nombre.strip().lower():
        return indice
return -1
```

La función compara los nombres uno a uno. lower permite buscar Luis escribiendo luis, y strip elimina espacios al comienzo y al final de la entrada. Se devuelve 1 para Luis y -1 si no hay coincidencia; el menú comprueba -1 antes de acceder a las listas y muestra Estudiante no registrado. No se eliminan tildes: los datos iniciales contienen Maria sin tilde. La búsqueda aplica recorrido y comparación.

8. Validación de índices

```
def leer_entero(mensaje, minimo, maximo):
    while True:
        try:
            valor = int(input(mensaje))
        except ValueError:
            print("Entrada incorrecta: escriba un número entero.")
            continue
        if minimo <= valor <= maximo:
            return valor
        print(f"Valor fuera de rango: debe estar entre {minimo} y {maximo}.")
```

Esta función solicita un entero y solo lo devuelve si se encuentra dentro de los límites indicados. Para consultar una nota se usa el rango de 0 a 4. Si escribo letras, se captura ValueError; si escribo 9, se muestra Valor fuera de rango y se solicita otra entrada. Así evito acceder a una posición inexistente y producir IndexError. El ciclo while permite corregir el dato sin cerrar el programa.

REPOSITORIO DEL PROYECTO

Nombre del proyecto: Sistema de Calificaciones en Python.

Repositorio GitHub: https://github.com/mrcarlos1990/sistema-calificaciones-python-capitulo-ix

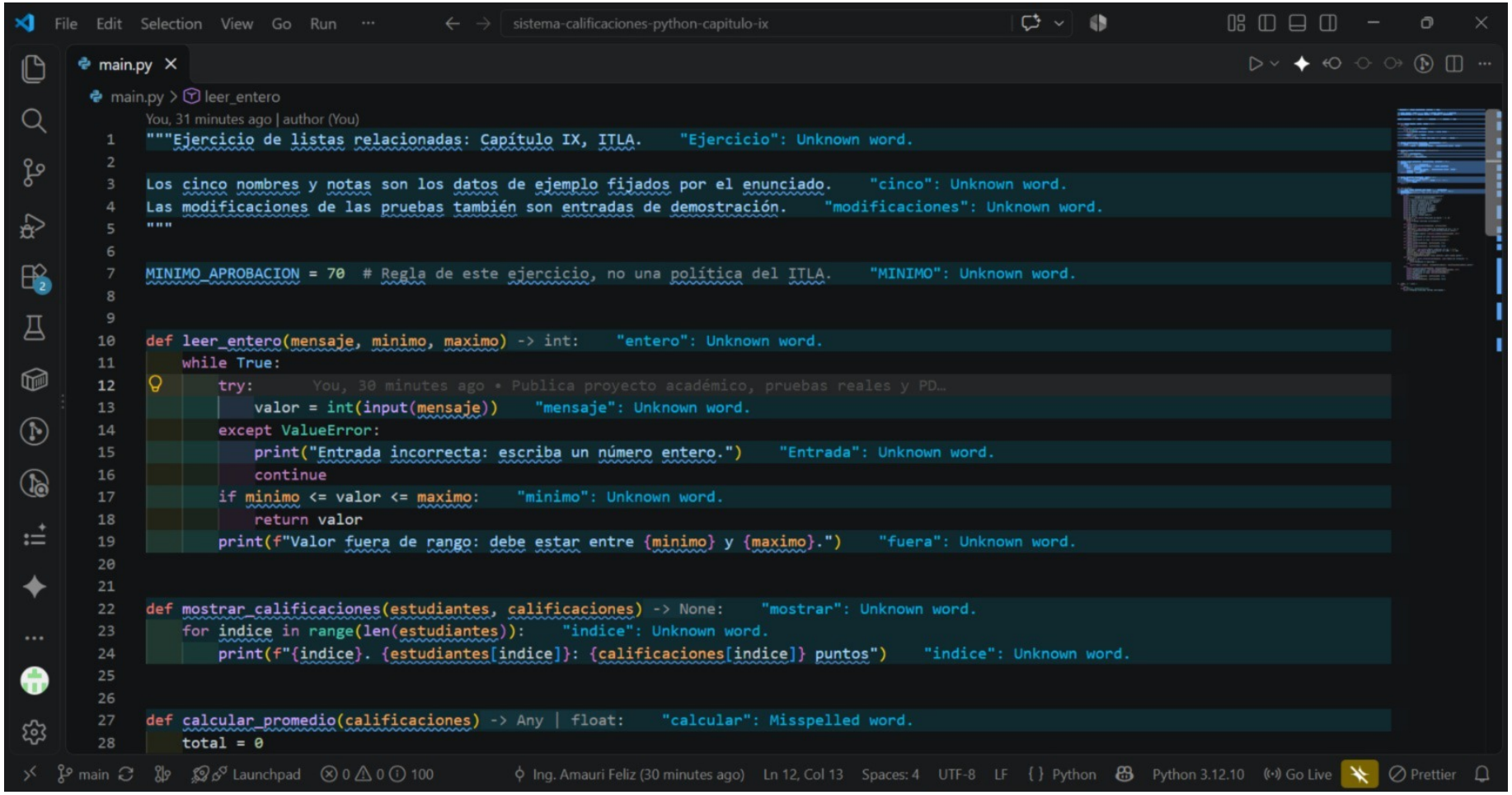
El repositorio es público y permite consultar el código completo de main.py. Incluye README.md con la descripción, el objetivo, las funcionalidades y las instrucciones de ejecución. En Windows puedo abrir la carpeta en Visual Studio Code y ejecutar py main.py en la terminal integrada.

El acceso público y la presencia de main.py y README.md se comprobaron el 10 de octubre de 2026. Esta revisión del PDF y las siete imágenes están preparadas localmente; todavía no sustituyen la documentación anterior publicada en GitHub.

Evidencias de ejecución

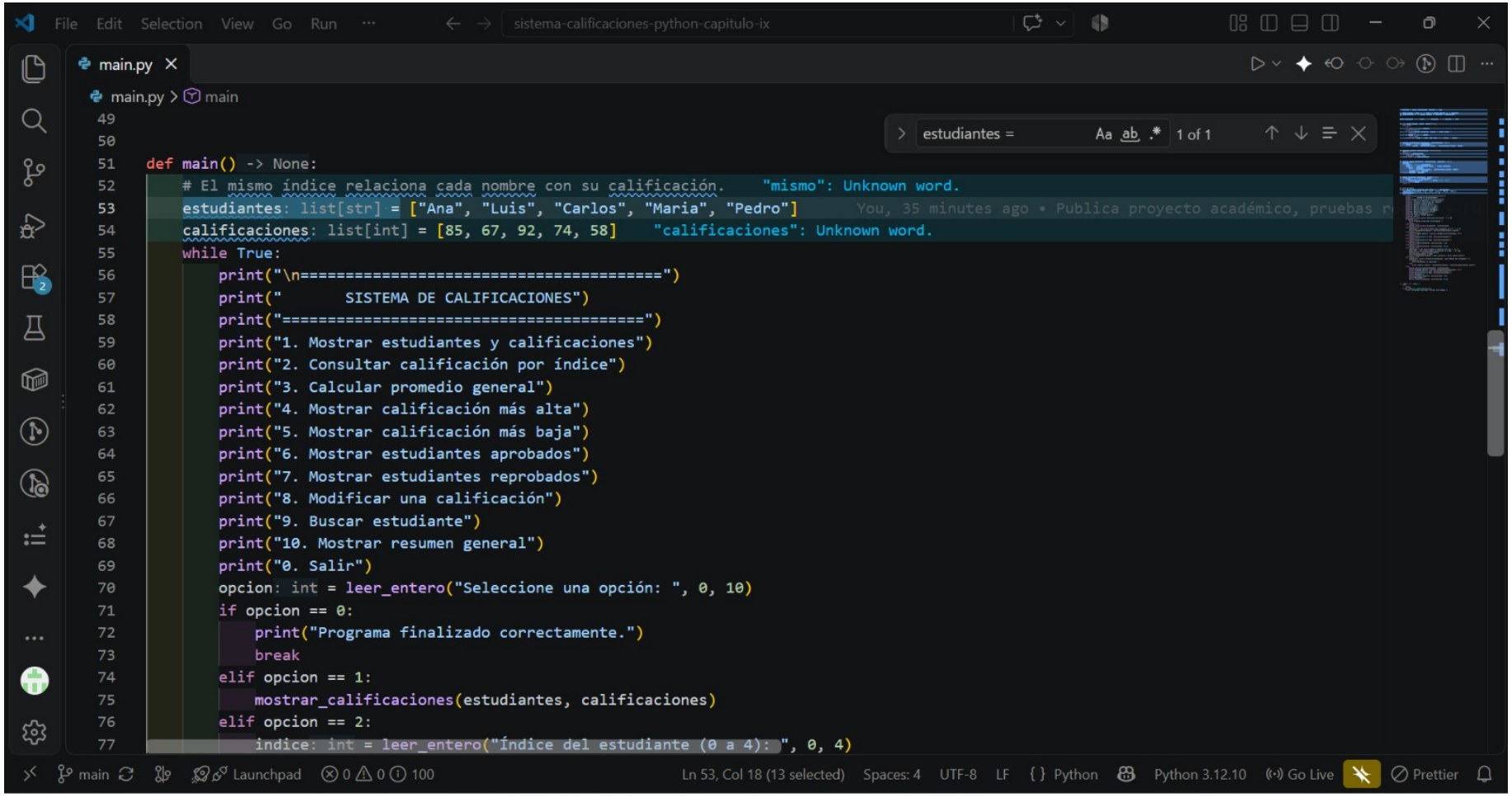
Las siete capturas reales aportadas por el estudiante se presentan completas y en orden, con sus títulos y explicaciones. Los avisos Unknown word son del corrector ortográfico del editor, no errores de ejecución. Las indicaciones de tipo atenuadas son ayudas visuales de VS Code. Además, once registros TXT conservan las entradas y salidas de las pruebas; resultados.json contiene sus comprobaciones. Las páginas horizontales facilitan la lectura de las figuras.

Figura 1. Código fuente desarrollado en Python.



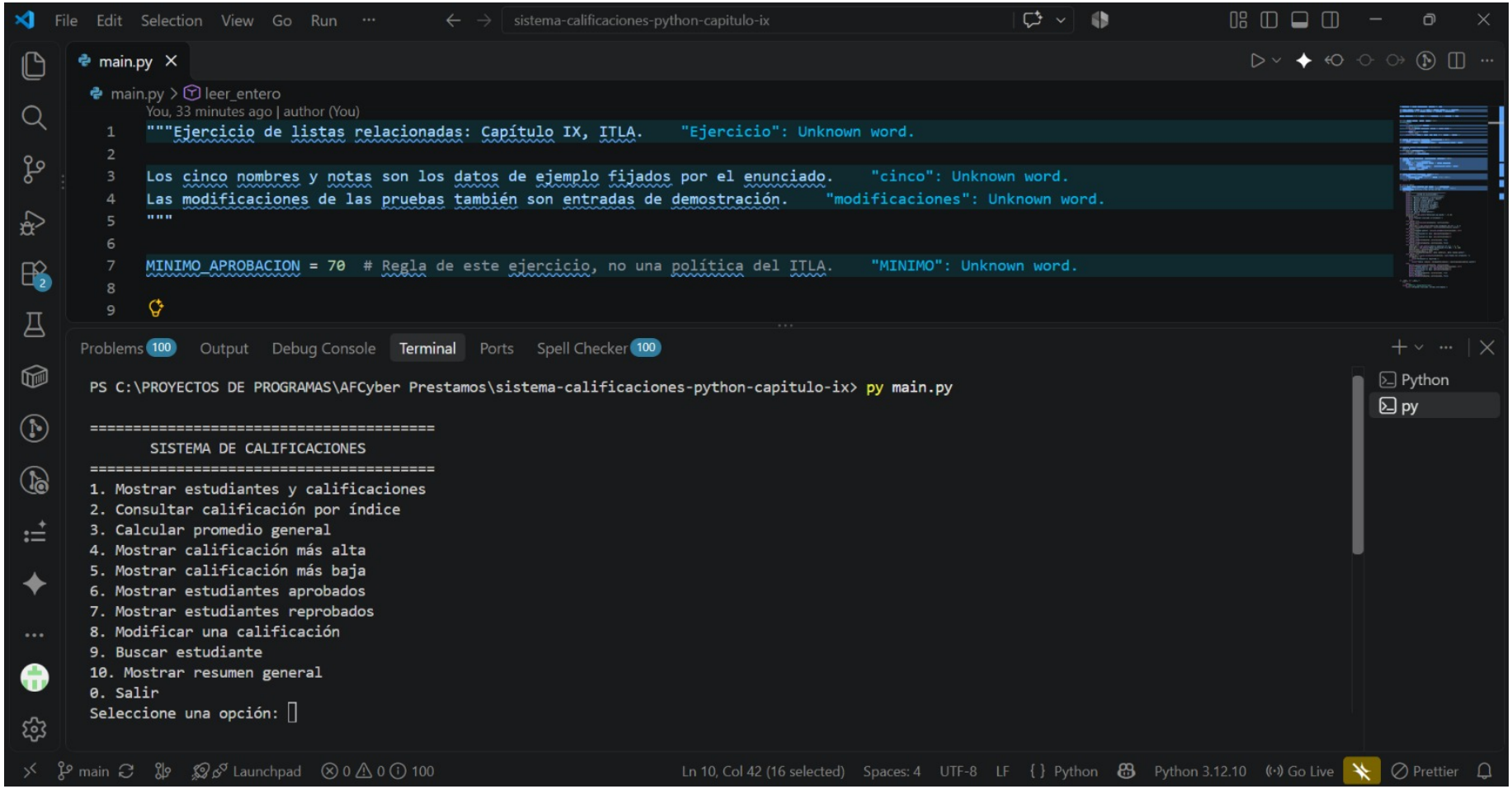
Presento main.py abierto en Visual Studio Code. La función leer_entero valida las entradas y el ciclo for permite mostrar las calificaciones.

Figura 2. Creación de los vectores.



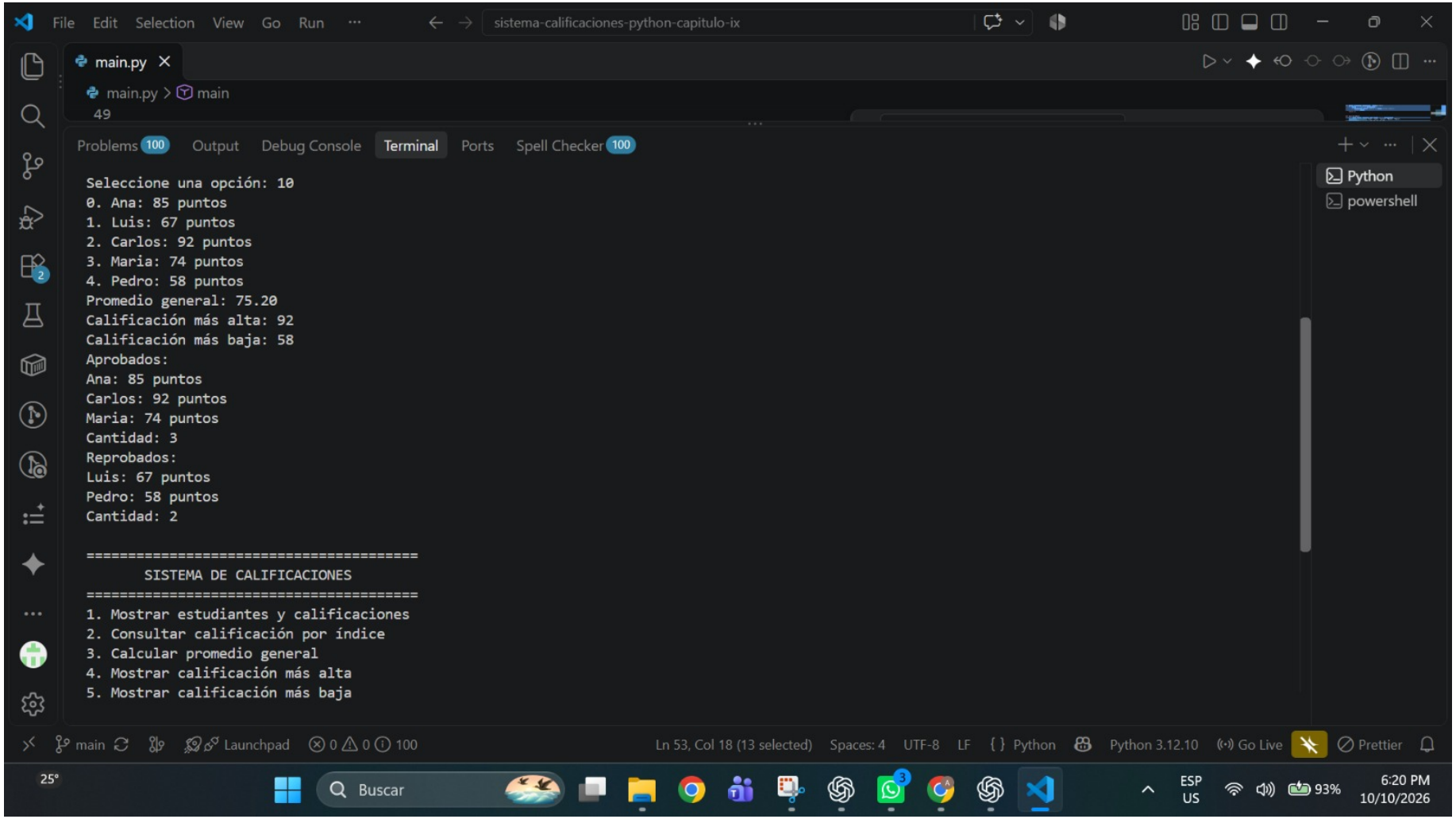
Utilizo dos listas de cinco elementos. Relaciono cada nombre con su nota mediante el mismo índice; por ejemplo, Ana ocupa la posición 0 y tiene 85 puntos.

Figura 3. Ejecución inicial del programa.



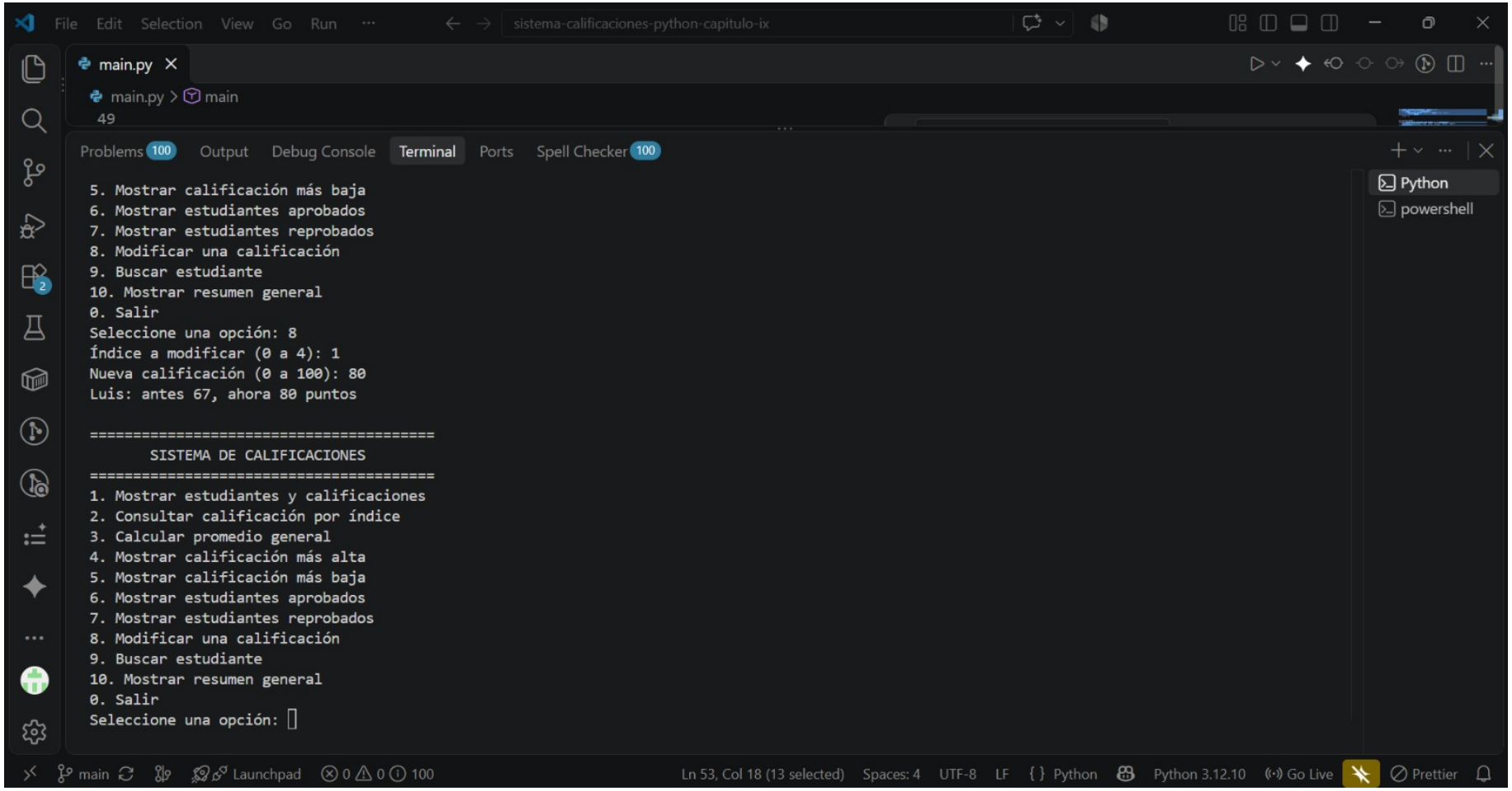
Inicio el programa desde la terminal integrada de Visual Studio Code. El menú ofrece diez operaciones y la opción 0 para salir.

Figura 4. Resultados generales de las calificaciones.



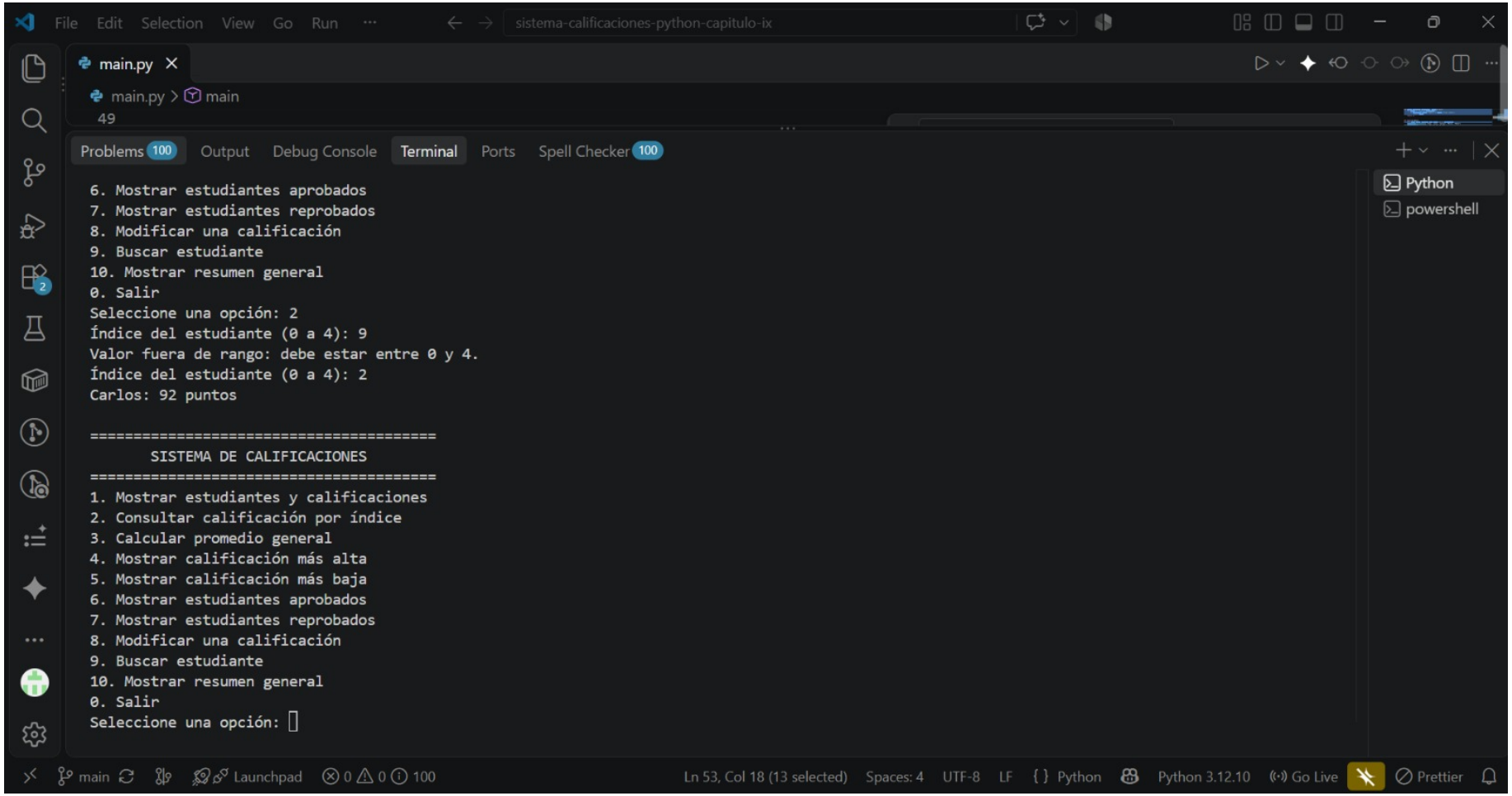
Consulto el resumen con los datos iniciales. Obtengo un promedio de 75.20, una nota máxima de 92 y una mínima de 58; hay tres aprobados y dos reprobados.

Figura 5. Modificación de una calificación.



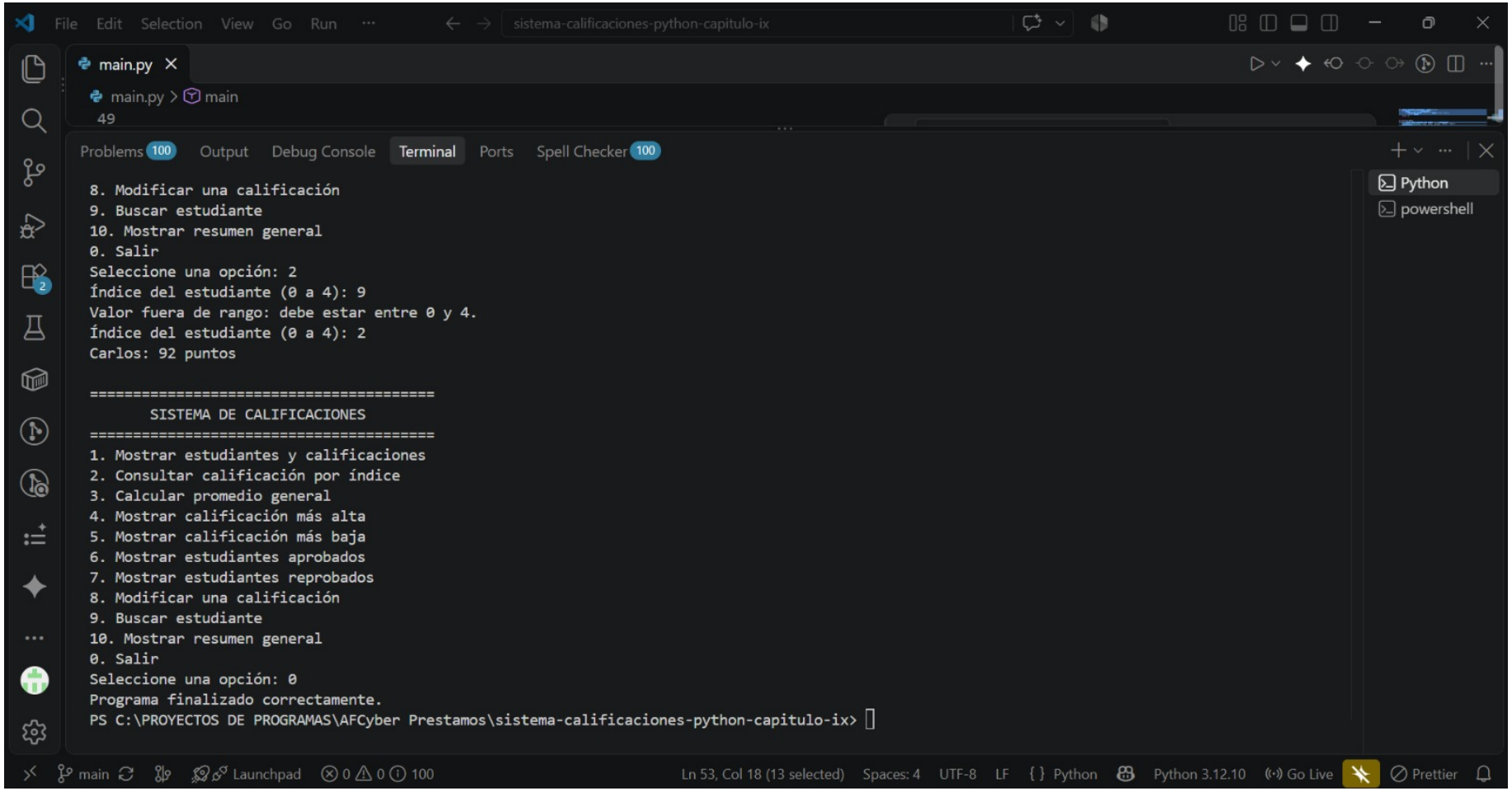
Selecciono la opción 8, el índice 1 y la nota 80. El mensaje confirma que la calificación de Luis cambió de 67 a 80 puntos y el menú continúa disponible.

Figura 6. Validación de índices incorrectos.



Introduzco el índice 9 y el programa lo rechaza. Al corregirlo a 2, consulto a Carlos con 92 puntos; la validación permite continuar sin cerrar el sistema.

Figura 7. Finalización correcta del programa.



Selecciono la opción 0. El mensaje Programa finalizado correctamente y el regreso a la terminal muestran que el ciclo del menú terminó de forma normal.

Casos de prueba

Se ejecutó python -m unittest discover -s pruebas -v con Python 3.12.14. Un método automatizado contiene once sesiones independientes. Cada sesión inicia el programa con las listas originales y termina con la opción 0. Los estados de la tabla se obtienen del archivo resultados.json generado por esa ejecución; no se asignan sin comprobar las salidas.

La modificación de Luis a 80 se comprueba dentro de una sola sesión. Las pruebas de aprobados y reprobados parten de los datos iniciales en otra sesión. La sesión 9 comprueba notas 0, 100 y el umbral exacto de 70. La sesión 10 verifica el resumen tras modificar a Luis y la 11 comprueba la salida directa con la opción 0. Los detalles íntegros, incluidas las entradas y las salidas completas, están en las transcripciones.

Prueba	Datos utilizados	Resultado esperado	Resultado obtenido	Estado
1. Visualización inicial	1; 0	0. Ana: 85; 1. Luis: 67; 2. Carlos: 92; 3. Maria: 74; 4. Pedro: 58	0. Ana: 85; 1. Luis: 67; 2. Carlos: 92; 3. Maria: 74; 4. Pedro: 58. Registro 01.	APROBADA
2. Promedio	3; 0	Promedio general: 75.20	Promedio general: 75.20. Registro 02.	APROBADA
3. Consulta por índice	2; 2; 0	Carlos: 92 puntos	Carlos: 92 puntos. Registro 03.	APROBADA
4. Modificación	8; 1; 80; 1; 3; 0	Luis: antes 67, ahora 80; 1. Luis: 80; Promedio general: 77.80	Luis: antes 67, ahora 80; 1. Luis: 80; Promedio general: 77.80. Registro 04.	APROBADA
5. Aprobados y reprobados	6; 7; 0	Ana: 85; Carlos: 92; Maria: 74; Cantidad: 3; Luis: 67; Pedro: 58; Cantidad: 2	Ana: 85; Carlos: 92; Maria: 74; Cantidad: 3; Luis: 67; Pedro: 58; Cantidad: 2. Registro 05.	APROBADA
6. Validación y continuidad	letras; 99; 2; 9; -1; 2; 8; 0; -5; 101; abc; 90; 1; 0	escriba un número entero; entre 0 y 10; entre 0 y 4; entre 0 y 100; Ana: antes 85, ahora 90; 0. Ana: 90	escriba un número entero; entre 0 y 10; entre 0 y 4; entre 0 y 100; Ana: antes 85, ahora 90; 0. Ana: 90. Registro 06.	APROBADA
7. Búsqueda	9; lUiS ; 9; Sofia; 0	Índice 1. Luis: 67; Estudiante no registrado.	Índice 1. Luis: 67; Estudiante no registrado.. Registro 07.	APROBADA
8. Extremos y resumen	10; 0	Promedio general: 75.20; Calificación más alta: 92; Calificación más baja: 58; Cantidad: 3; Cantidad: 2	Promedio general: 75.20; Calificación más alta: 92; Calificación más baja: 58; Cantidad: 3; Cantidad: 2. Registro 08.	APROBADA
9. Límites y aprobación en 70	8; 0; 0; 8; 1; 100; 8; 4; 70; 10; 0	Ana: antes 85, ahora 0; Luis: antes 67, ahora 100; Pedro: antes 58, ahora 70; Promedio general: 67.20; Cantidad: 4; Cantidad: 1	Ana: antes 85, ahora 0; Luis: antes 67, ahora 100; Pedro: antes 58, ahora 70; Promedio general: 67.20; Cantidad: 4; Cantidad: 1. Registro 09.	APROBADA

Prueba	Datos utilizados	Resultado esperado	Resultado obtenido	Estado
10. Resumen después de modificar a Luis	8; 1; 80; 10; 0	Luis: antes 67, ahora 80; 1. Luis: 80 puntos; Promedio general: 77.80; Calificación más alta: 92; Calificación más baja: 58; Cantidad: 4; Cantidad: 1	Luis: antes 67, ahora 80; 1. Luis: 80 puntos; Promedio general: 77.80; Calificación más alta: 92; Calificación más baja: 58; Cantidad: 4; Cantidad: 1. Registro 10.	APROBADA
11. Salida con opción 0	0	Programa finalizado correctamente.	Programa finalizado correctamente.. Registro 11.	APROBADA

Conclusión

¿Qué aprendí sobre los arreglos?

Aprendí que puedo reunir varios datos bajo un mismo nombre y trabajar con ellos por sus posiciones. En este ejercicio utilicé listas de Python para representar vectores: una guarda los nombres y otra las calificaciones.

¿Por qué son importantes los índices?

Los índices me permiten localizar, consultar y modificar un elemento específico. En las dos listas del programa, el mismo índice relaciona a un estudiante con su nota. Comprendí que la primera posición es 0 y que cinco elementos tienen como último índice 4.

¿Qué ventajas ofrecen los vectores?

Los vectores me ayudan a organizar varios valores y a procesarlos sin crear una variable independiente para cada estudiante. Puedo recorrer las cinco calificaciones con un ciclo y aplicar la misma operación a cada una.

¿Cómo se relacionan los ciclos con los arreglos?

El ciclo for permite visitar los elementos para mostrarlos, sumarlos o buscar un nombre. El ciclo while del sistema mantiene el menú disponible y repite la solicitud cuando la entrada no es válida. Las condiciones seleccionan las opciones y permiten identificar quién cumple el mínimo de 70 puntos definido para este ejercicio.

¿Qué dificultad encontré desarrollando el programa?

Una dificultad que tuve fue comprender cómo funcionan los índices en Python, porque las posiciones comienzan desde cero y no desde uno. También tuve que prestar atención a lo que sucedía cuando se introducía un índice que no existía en la lista.

¿Cómo solucioné esa dificultad?

Para resolverlo, revisé el código, realicé diferentes pruebas y utilicé una validación que detecta valores fuera del rango establecido. Así comprendí mejor cómo acceder, consultar y modificar los elementos de un arreglo sin que el programa se cierre por un índice incorrecto.

¿En qué aplicación real podría utilizar arreglos?

Podría utilizarlos para organizar calificaciones, cantidades de productos en un inventario o registros de ventas. En el sistema desarrollado apliqué listas, índices, asignaciones, ciclos, condiciones, operaciones matemáticas, búsqueda y validación. El ejemplo funciona durante una sesión de consola y me permite explicar cómo se relacionan esos conocimientos para resolver un problema sencillo.

Fuentes consultadas

Python Software Foundation. Una introducción informal a Python, sección Listas. Documentación oficial. https://docs.python.org/es/3/tutorial/introduction.html. Consulta: 10 de octubre de 2026. Recurso utilizado para verificar listas, acceso por posición y modificación.

Python Software Foundation. Estructuras de datos. Tutorial oficial de Python. https://docs.python.org/es/3/tutorial/datastructures.html. Consulta: 10 de octubre de 2026. Referencia para las operaciones sobre listas.

Python Software Foundation. Más herramientas para control de flujo. Tutorial oficial de Python. https://docs.python.org/es/3/tutorial/controlflow.html. Consulta: 10 de octubre de 2026. Referencia para for, condiciones y control de ciclos.

Las explicaciones y los ejemplos del proyecto se redactaron para esta asignación. Las referencias respaldan el comportamiento del lenguaje y no se reproducen definiciones extensas de las páginas consultadas.